`WHERE SALARY LIKE '%2'`

Finds any values that end with 2.

`WHERE SALARY LIKE '_2%3'`

Finds any values that have a 2 in the second position and end with a 3.

`WHERE SALARY LIKE '2___3'`

Finds any values in a five-digit number that start with 2 and end with 3.

_ underscore wildcard

To understand '_' wildcard let us go back to the movie database example. The wildcard underline fits exactly one word. Let's say we're searching for all the films released in the 200x years, where x is just one character that can have some meaning. To do that, we would use the wild card. The corresponding script lists all the films released in "200x"

```
SELECT * FROM movies WHERE year_released LIKE '200_';
```

Running the above script against the myflix db in MySQL workbench provides us with the following results.

movie_id; title; director; year_released; category_id

2	9
Forgetting Sarah Marshall	Honeymooners
Nicholas Stoller	John Shultz
2008	2005
2	8

NOT Like

In conjunction with wildcards, the logical NOT operator can be used to return lists, which do not correspond to the pattern. Suppose we want films that have not been released in the 200x year. Together with the underscore, we would use the logical NOT operator to achieve our results.

```
SELECT * FROM movies WHERE year_released NOT LIKE '200_';
```

movie_id; title; director; year_released; category_id

1	4
Pirates of the Caribbean 4	Code Name Black
Rob Marshall	Edgar Jimz
2011	2010
1	NULL

Remember that in our result collection only films that do not begin with 200 in the year they are released. That's because in our wildcard sequence check we used the NOT logical operator.